Docker: Zero to Hero

The Complete Masterclass—from "It works on my machine" to production-ready container orchestration.



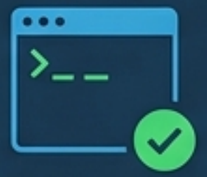
Gemini Notebook

The Syllabus



Checkpoint 1: Core Concepts

(Images, Containers, VMs vs. Docker)



Checkpoint 2: The Command Toolkit

(Lifecycle, Ports, Env Vars, Troubleshooting)



Checkpoint 3: Building & Publishing

(Dockerfiles, Image Layers, Docker Hub)



Checkpoint 4: Advanced Orchestration

(Networks, Volumes, Docker Compose)

Gemini Notebook

The Problem

“It works on my machine!”
Manual dependency installation causes version conflicts and environment mismatch across teams.



The Solution

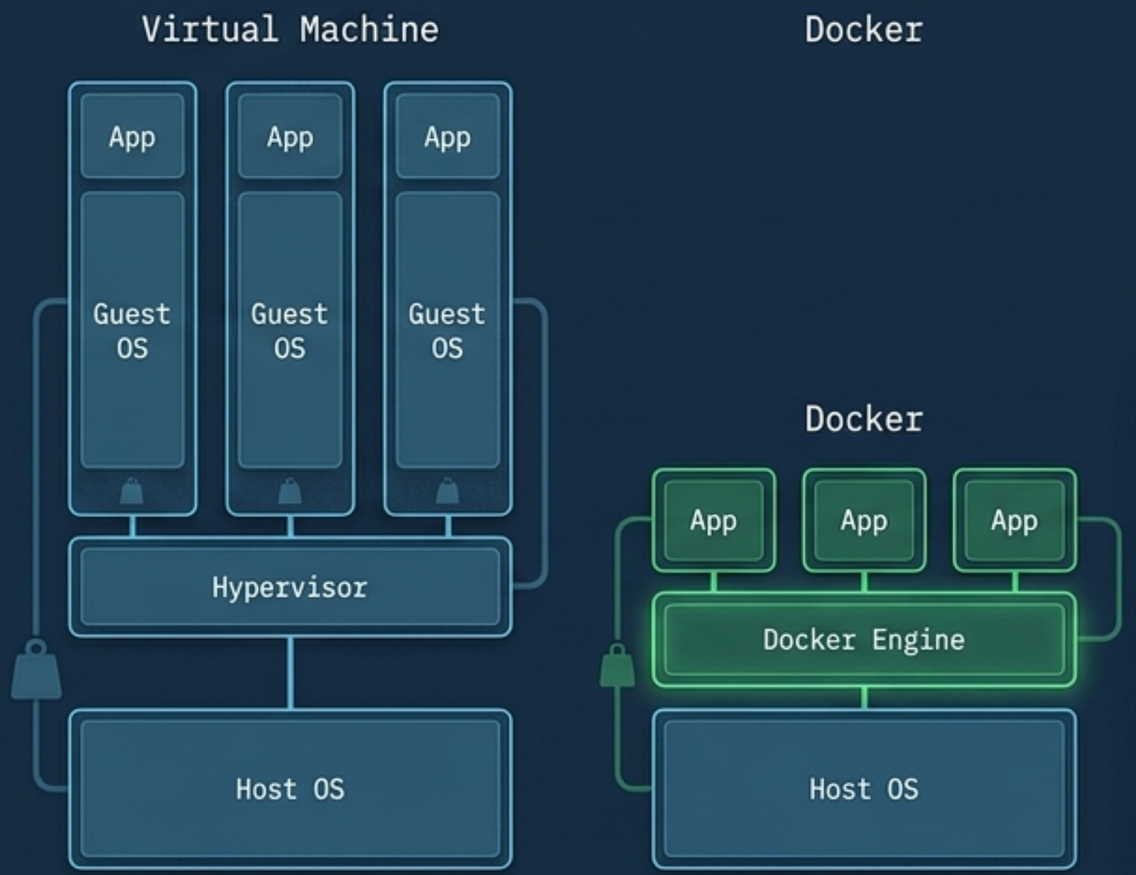
Docker. It bundles the application and its underlying dependencies into a single, standardized, lightweight unit (a container) that runs identically everywhere.



Gemini Notebook

Docker vs. Virtual Machines

Architectural Diagrams



Feature	Virtual Machine	Docker
Resource Usage	Heavy, GBs of RAM	Lightweight, shares host OS kernel
Startup Time	Minutes	Milliseconds
Virtualization	Hardware-level, full OS isolation	Application-level, isolated processes
Portability	Tied to specific hypervisors	Highly portable across any OS with Docker

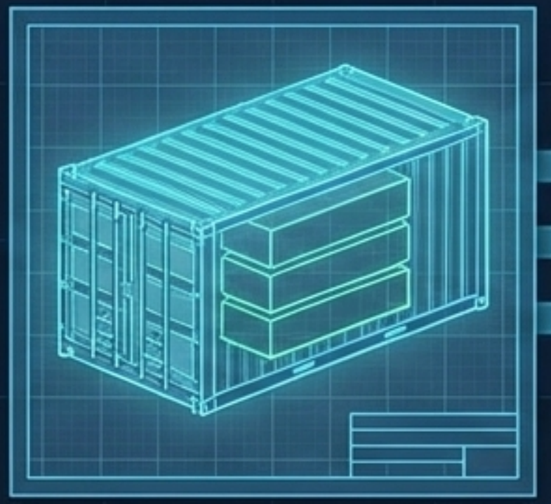
Gemini Notebook

The Mental Model: Images vs. Containers

Dockerfile
(The Instructions)

```
FROM node:16
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "server.js"]
```

Docker Image
(The Blueprint)



A read-only, static snapshot containing the app and all instructions.
(Comparable to an OOP Class).

Docker Container
(The Instance)

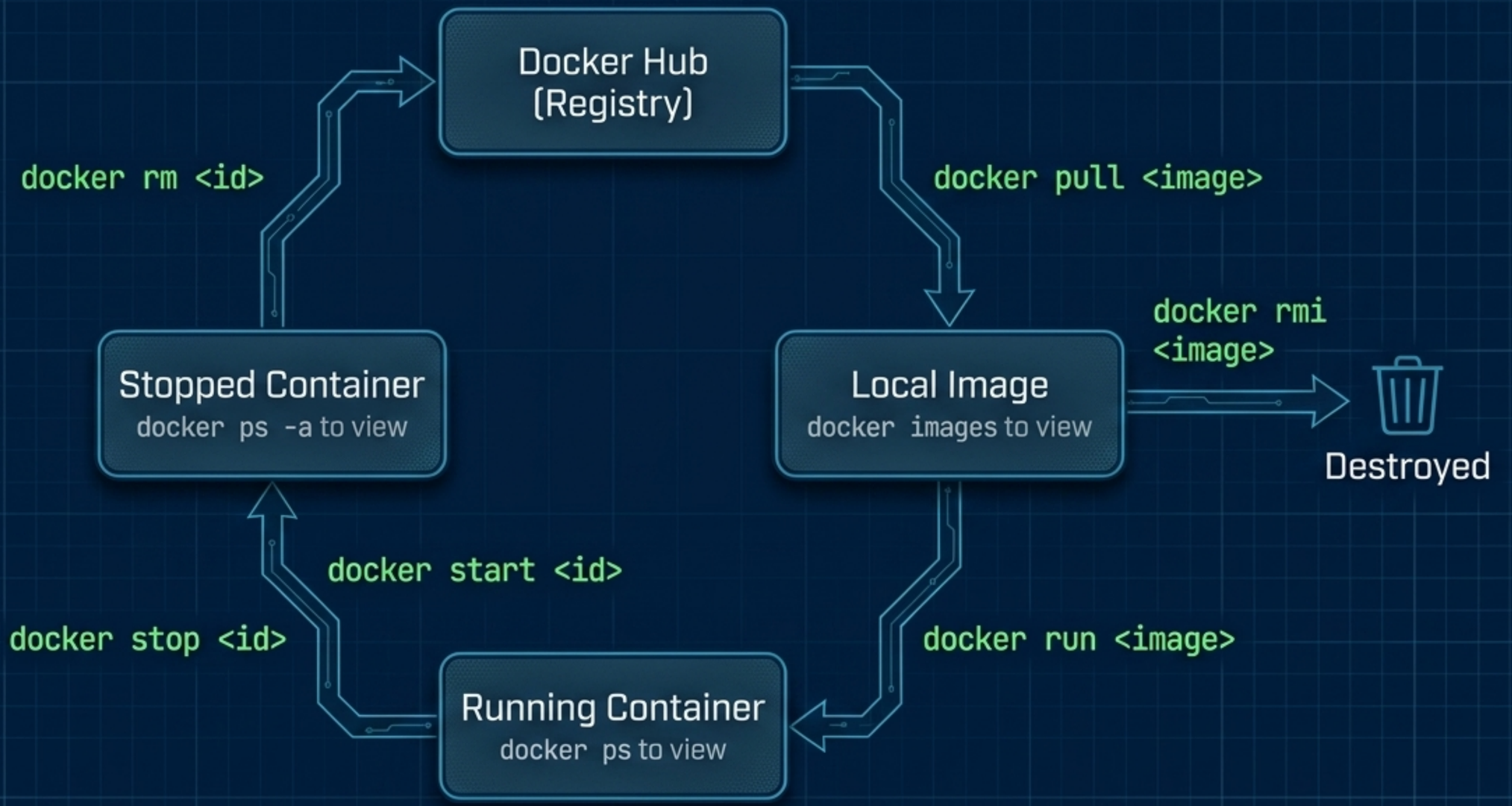


A running, isolated instance of an image that uses system resources like CPU and Memory.
(Comparable to an OOP Object).

Key Insight: A single Image can spawn hundreds of identical Containers.

Gemini Notebook

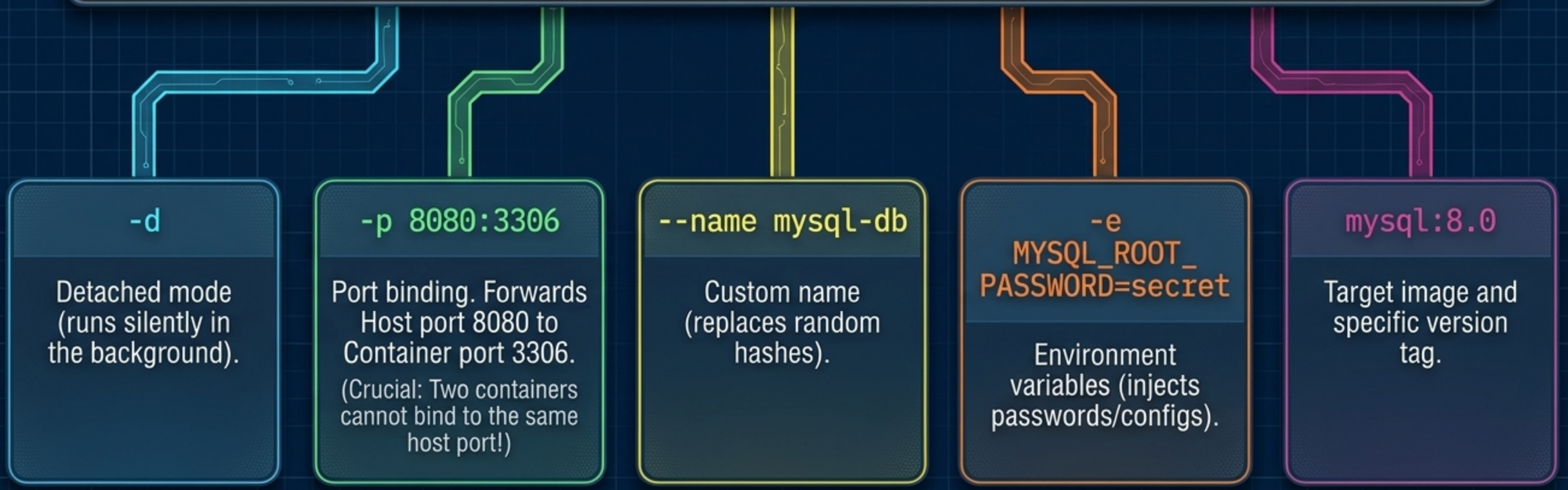
The Essential Command Map



Gemini Notebook

Demystifying docker run

```
docker run -d -p 8080:3306 --name mysql-db
-e MYSQL_ROOT_PASSWORD=secret mysql:8.0
```



Gemini Notebook

Troubleshooting & Inside the Container

Live Diagnostics

```
2023-10-27 10:00:01.123 UTC [Note] mysqld (mysqld 8.0.31) starting...
2023-10-27 10:00:01.345 UTC [Note] InnoDB: The InnoDB memory heap is disabled
2023-10-27 10:00:01.567 UTC [Note] InnoDB: Mutexes and rw_locks use GCC atomic
builtins
2023-10-27 10:00:01.789 UTC [Note] InnoDB: Compressed tables use zlib 1.2.11
2023-10-27 10:00:02.012 UTC [Note] InnoDB: Using Linux native AIO
2023-10-27 10:00:02.234 UTC [Note] InnoDB: Initializing buffer pool, total size =
2023-10-27 10:00:02.456 UTC [Note] InnoDB: Completed initialization of buffer pool
2023-10-27 10:00:02.678 UTC [Note] InnoDB: 128 transaction(s) which must be rolled
2023-10-27 10:00:02.901 UTC [Note] InnoDB:
2023-10-27 10:00:03.123 UTC [Note] InnoDB: Rolling back trx with id 1234567
2023-10-27 10:00:02.123 UTC [Note] InnoDB: Rolling back trx with id 1234567
2023-10-27 10:00:03.345 UTC [Note] InnoDB: Rolling back trx with id 1234568
2023-10-27 10:00:03.567 UTC [Note] InnoDB: Rolling back trx with id 1234569
2023-10-27 10:00:03.789 UTC [Note] InnoDB: Rolling back trx with id 1234570
2023-10-27 10:00:04.012 UTC [Note] InnoDB: Rolling back trx with id 1234571
```

`docker logs -f <container>`

Fetches and follows live terminal output. Essential for debugging crashes or verifying startup (e.g., MySQL initialization).

Deep Access

`docker exec -it <container> /bin/bash`

Opens an interactive terminal inside the running container to view files or test configurations.



`docker inspect <container>`

Returns detailed JSON with internal IP addresses, networks, mounts, and config data.

Gemini Notebook

Dockerizing an App (The Dockerfile & Layers)

```
FROM node
ENV MONGO_DB_USERNAME=admin
RUN mkdir -p docker-testapp
COPY . /docker-testapp
CMD ["node", "/docker-testapp/server.js"]
```



Startup Command

Source Code Layer
(Changes often)

Execution Layer

Config Layer

Base Image Layer
(Heavy, immutable)

The **Caching Insight:** Docker caches each layer. If only your source code changes, Docker instantly reuses the base OS and Node layers, making subsequent `'docker build -t testapp:1.0 .'` commands incredibly fast.

Gemini Notebook

Real-World Architecture
(The Need for Networks & Volumes)



The Setup:

A 3-tier microservice architecture (Node.js + Mongo + Mongo-Express UI).

The Isolation Problem:

By default, isolated containers cannot securely talk to each other without exposing ports to the host system.

The Data Problem:

If the Mongo container crashes or is stopped, its internal virtual file system is wiped. All user data is permanently deleted.

The Next Step: We must build a Docker Network for secure communication and mount a Docker Volume to persist the data.

Gemini Notebook

Connecting Containers: Docker Networks



Bridge Network

Allows containers on the same host to communicate securely. A custom bridge enables automatic DNS resolution (containers can ping each other simply by their names).

```
docker network create mongo-network
```

Host Network

Removes isolation; the container shares the host's IP/ports directly.

Null Network

Complete network isolation for high security.

Gemini Notebook

The Golden Rule: Containers are disposable; data must be persistent.

Data Persistence: Docker Volumes

Without Volumes



Without Volumes: If the Mongo container is destroyed, the internal × virtual file system is wiped clean.

With Volumes



With Volumes: Data is written directly to the host machine. If the container dies, the data lives on safely.

Named Volumes: Managed internally by Docker. Best for production environments.

```
Command: docker volume create my-vol
```

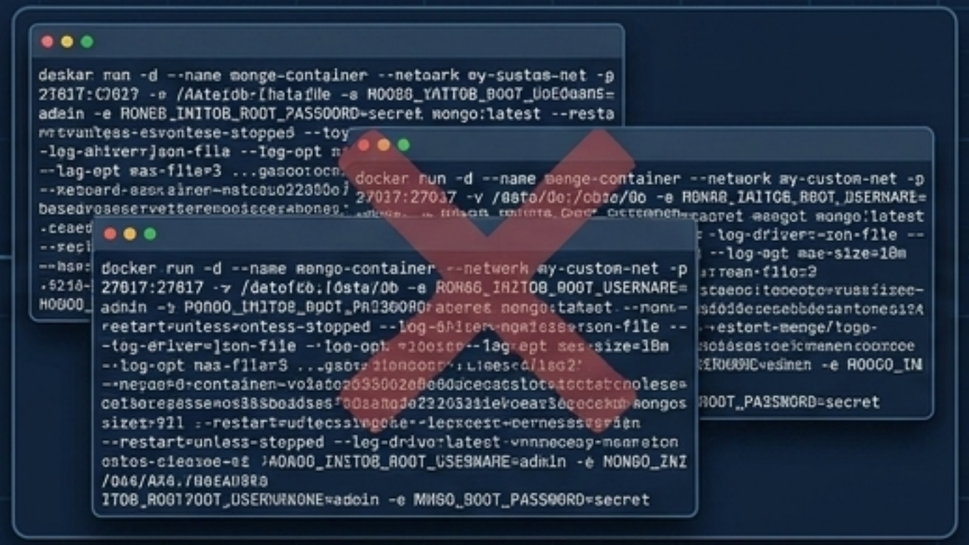
Bind Mounts: Maps a specific host folder directly to a container folder. Best for local development.

```
Example: -v /home/data:/data/db
```

Gemini Notebook

The Aha! Moment: Docker Compose

Manual Commands



Infrastructure as Code

```yaml
version: '3.8'
services:
  mongo:
    image: mongo
    ports:
      - '27017:27017'
    volumes:
      - /data:/data/db
```

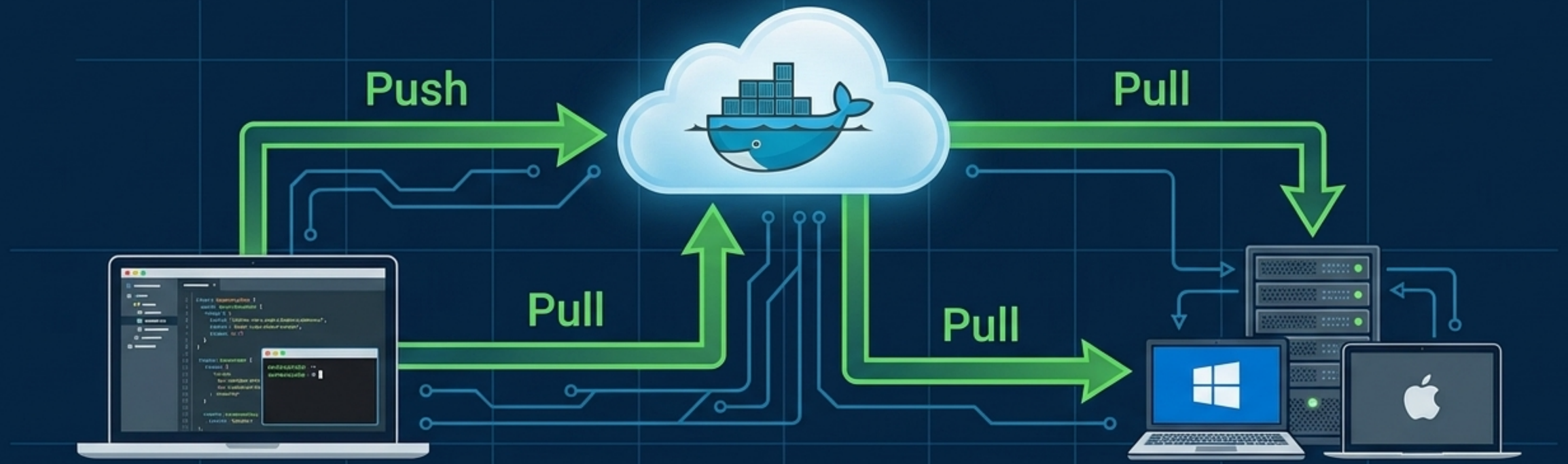
What is Docker Compose? A tool for defining and running multi-container architectures via a single YAML file.

The Magic: Automatically handles custom network creation and volume mapping.

The Commands: `docker compose -f mongodb.yaml up -d` (Start everything) and `docker compose down` (Tear it all down cleanly).

Gemini Notebook

Docker Hub (The App Store for Developers)



1 Authenticate

```
docker login
```

2 Prepare

```
docker tag testapp user/testapp
```

3 Distribute

```
docker push user/testapp
```

4 Deploy Anywhere

```
docker pull user/testapp
```

Result: Anyone can now run your app securely anywhere in the world without natively installing Node, Python, or Mongo!

Gemini Notebook

The Hero Summary

The Backend Developer's Master Checklist



1. Image vs. Container: Blueprint vs. Running Instance.



2. The `Dockerfile`: The reproducible recipe to build custom image layers.



3. Port Mapping (`-p host:container`): Controlling how the outside world talks to your app.



4. Volumes: Ensuring database persistence when containers die.



5. Docker Compose: Orchestrating complex microservices effortlessly with YAML.

Mastering these 5 concepts covers 95% of daily software engineering needs.

APNA COLLEGE
Gemini Notebook